



Software Design & Analysis Project Report

Course: Software Design And Analysis

Instructor Name: Jibran Rasheed Khan

Project: Nova - Hospital Management System

Submitted by: Yash Raj & Jiyanshu Raj

Roll number: 24k-0737 & 24k-0987

Date: 29-04-2025

1. Project Overview

1.1 Project Idea

NovaCare is a comprehensive, web-based Hospital Management System developed using the Django framework (Python) with a relational SQLite3 database backend. The system unifies the core operational workflows of a modern hospital into a single, integrated digital platform: patient registration and appointment management, clinical consultation records, and a full-featured pharmacy with e-commerce and billing capabilities.

1.2 Problem Statement

Hospitals and clinics in developing regions continue to rely on paper-based records, fragmented spreadsheets, and manual billing processes. This leads to:

- Appointment scheduling conflicts and duplicate bookings
- Loss or inaccessibility of patient medical history and prescriptions
- Inventory shrinkage and stock-out situations in the pharmacy
- Delayed and inaccurate billing that erodes patient trust
- No consolidated view for administrators to manage doctors and services

NovaCare directly addresses each of these pain points through automated workflows, centralised records, and real-time stock management.

1.3 Domain / Application Area

- Healthcare Information Systems (HIS)
- Hospital & Clinic Administration
- Pharmacy Management & E-Commerce
- Electronic Medical Records (EMR)

1.4 Target Users (Beneficiaries)

Patients	Book appointments, browse the medicine catalogue, manage a shopping cart, and receive a unified bill combining consultation fees and pharmacy charges.
Doctors	View their appointment schedule, approve or cancel visits, and add full consultation notes including diagnosis, prescription, and test report remarks.
Pharmacists	Manage medicine inventory (CRUD), receive low-stock alerts, process over-the-counter POS bills, and track sales history.
Admin / Staff	Add, edit, and delete doctor profiles; manage hospital departments and available services through a staff-restricted admin panel.

3. Functionalities and Features

NovaCare is designed around four independent yet deeply integrated modules. Below is a detailed breakdown of its functionalities and what distinguishes it from basic hospital applications.

3.1 Public-Facing Portal

- **Home Page:** Showcases hospital departments, hero banner, and quick navigation links
- **Doctor Directory:** Searchable and filterable listing of all registered doctors by name or department
- **Services Page:** Displays available hospital services (MRI, X-Ray, Blood Tests, etc.) with descriptions
- **Public Pharmacy:** Patients and visitors can search and browse the medicine catalogue without logging in
- **Branches & Contact:** Information pages for hospital locations and contact forms

3.2 Authentication & Role Management

- **Self-Registration:** Patients register via a form; a Patient profile is automatically created and linked
- **Role-Based Login:** A single login endpoint redirects users to their respective dashboard based on role flags
- **Django Admin:** Staff and superusers access the full admin panel for master data management
- **Secure Logout:** Session clearing with CSRF protection on all state-changing operations

3.3 Appointment Management

- **Smart Booking:** Patients select a doctor, date, and time; the system enforces slot uniqueness to prevent double-booking
- **Doctor Approval:** Doctors confirm or cancel appointments directly from their dashboard
- **Consultation Notes:** Doctors record full medical notes per appointment: diagnosis, prescription, and test report remarks
- **Status Lifecycle:** Appointments progress through Pending → Confirmed → Completed → (or Cancelled)
- **Medical History:** Doctors can view a complete chronological history of completed appointments for any patient

3.4 Pharmacy & Billing

- **Inventory CRUD:** Pharmacists add, edit, and delete medicines with formula, price, stock, expiry, and prescription flags
- **Low-Stock Alerts:** Dashboard automatically flags medicines with stock below 10 units
- **POS Billing (OTC):** Walk-in customers are billed via a point-of-sale interface; stock is automatically deducted
- **E-Commerce Cart:** Patients add medicines to a persistent cart with live stock validation and quantity management
- **Unified Checkout:** A single checkout combines pending consultation fees (doctor's fee) and cart medicines into one bill
- **Expenditure History:** Patients view all past UnifiedBills with itemised cost breakdowns
- **Automatic Reconciliation:** On checkout: stock is deducted, cart is cleared, and all pending appointments are marked as paid

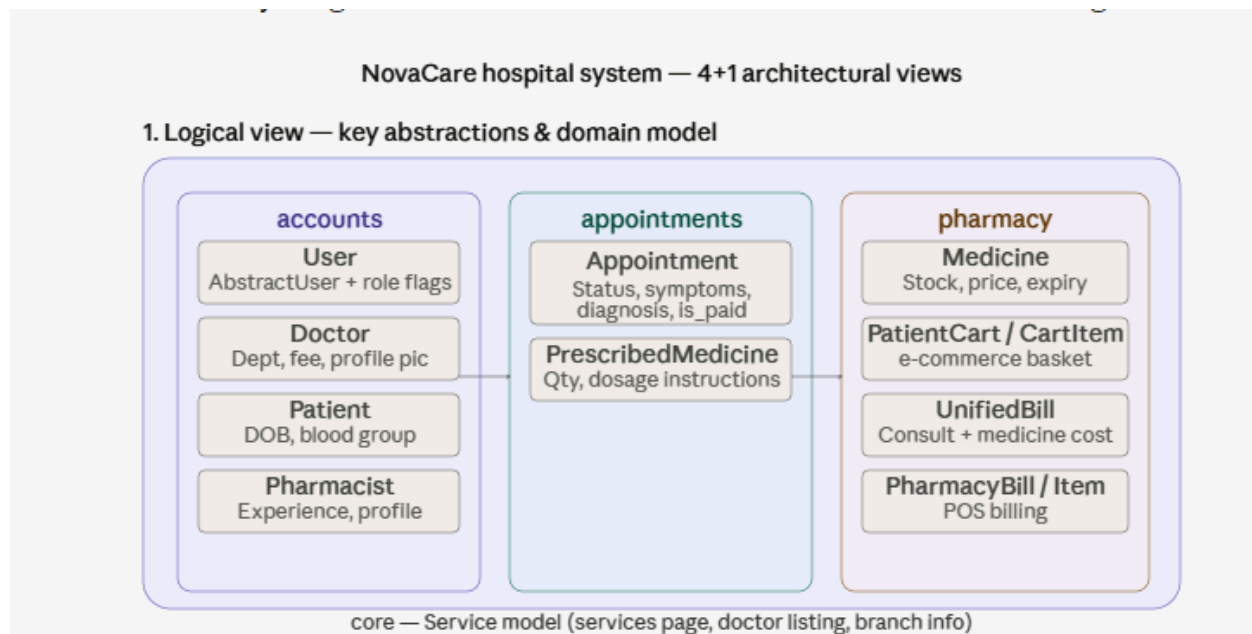
What Makes NovaCare Unique: Unlike standalone appointment or pharmacy apps, NovaCare's Unified Billing Engine consolidates consultation and medicine charges in a single transaction, eliminating the need for patients to interact with separate billing counters. The role-based architecture ensures every user sees only the features relevant to their workflow.

4. SYSTEM DESIGN - UML DIAGRAMS

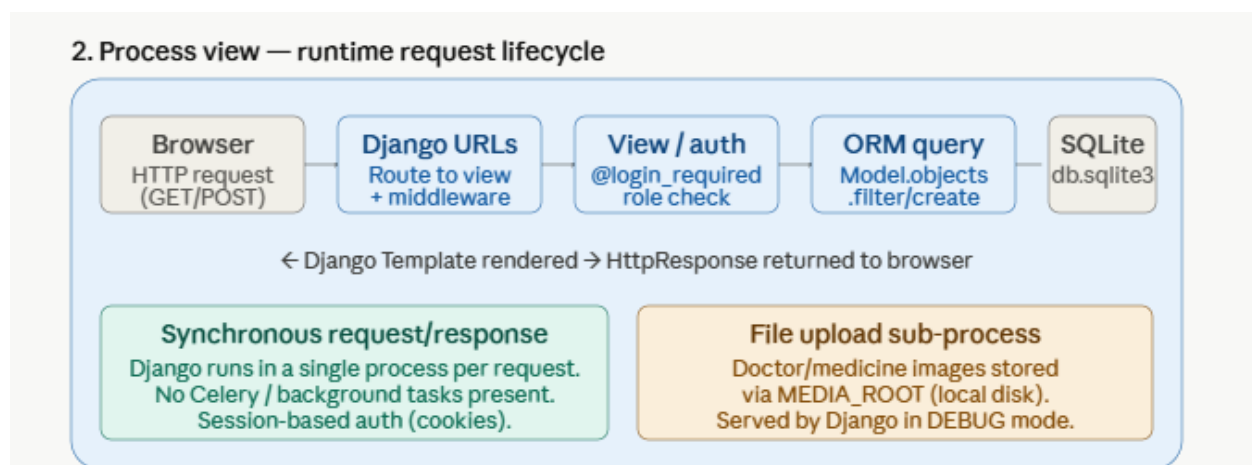
- **4+1 View Model:**

Purpose: The 4+1 View Model is used to describe a system's architecture from multiple perspectives so that different stakeholders (developers, users, managers, testers) can all understand it clearly.

1. Logical View

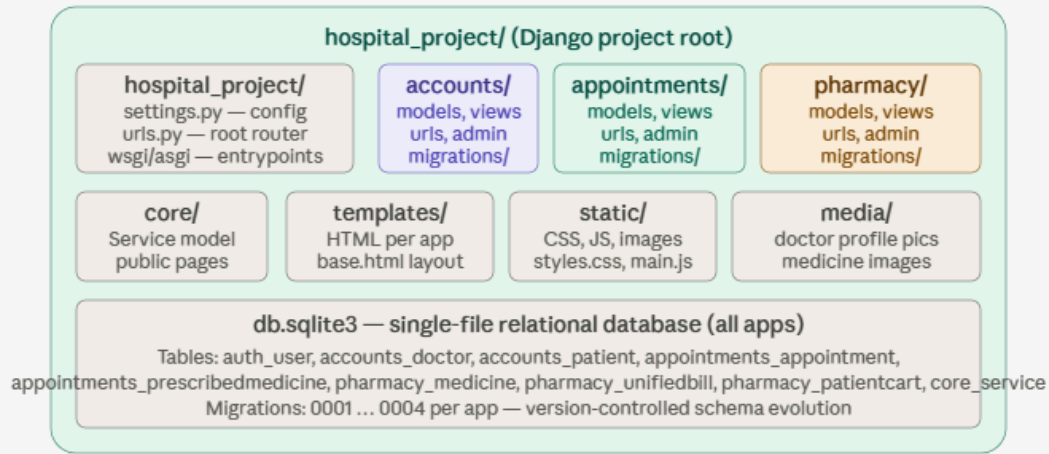


2. Process View



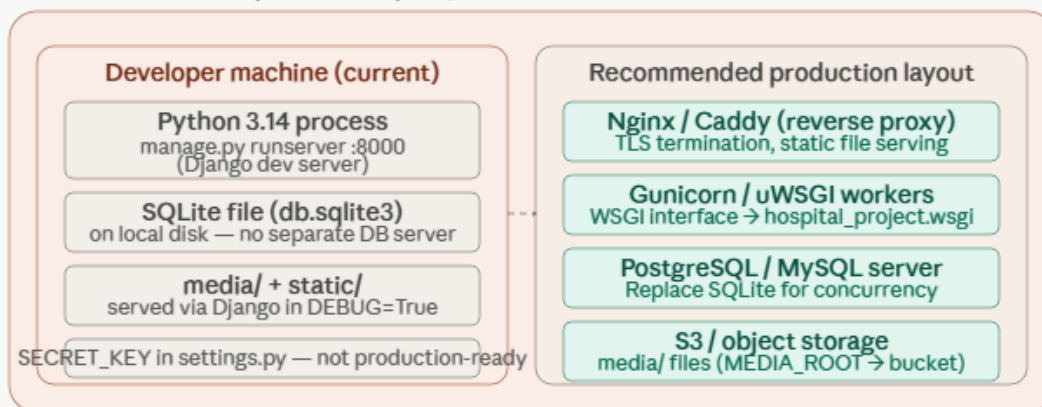
3. . Development View (Implementation View)

3. Development view — module & package structure



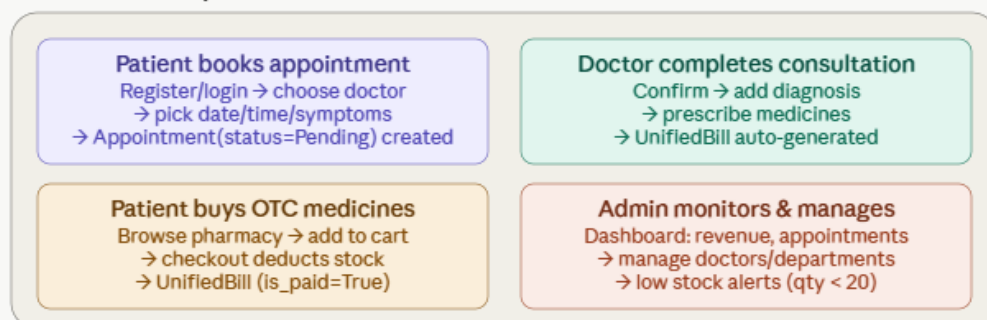
4. Physical View (Deployment View)

4. Physical view — deployment topology



5. +1 Scenario View (Use Case View)

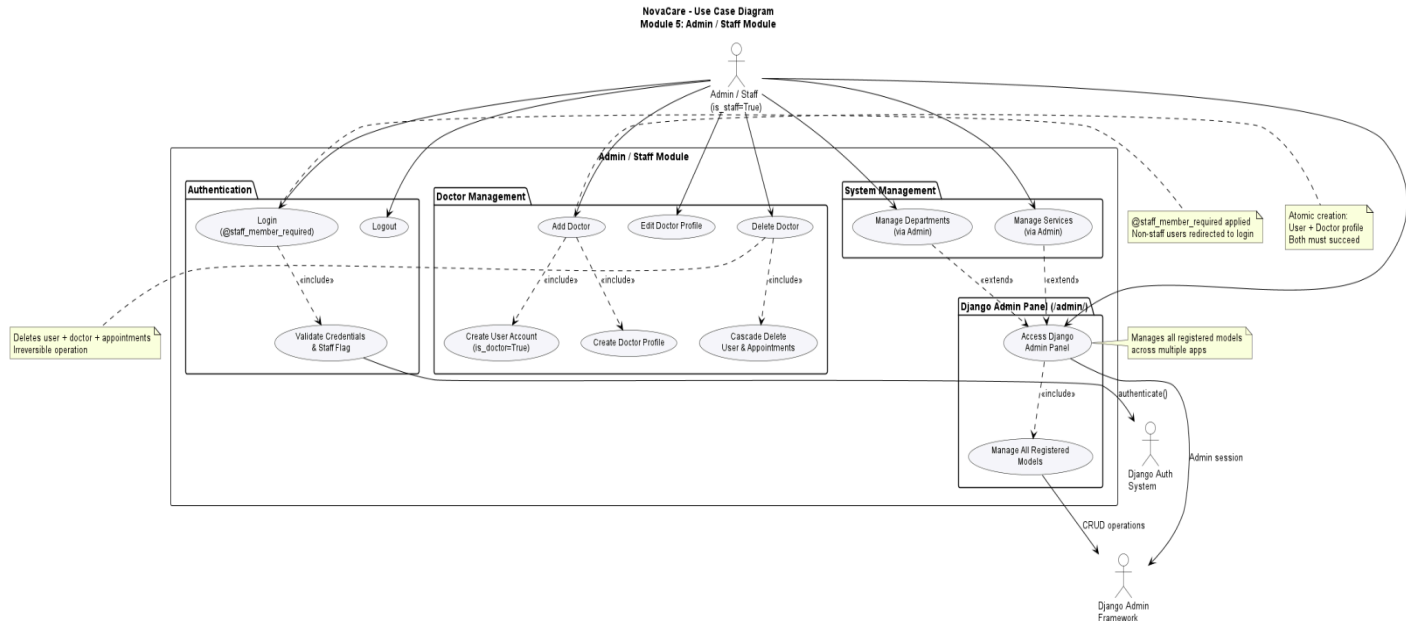
5. Scenarios — key end-to-end use cases



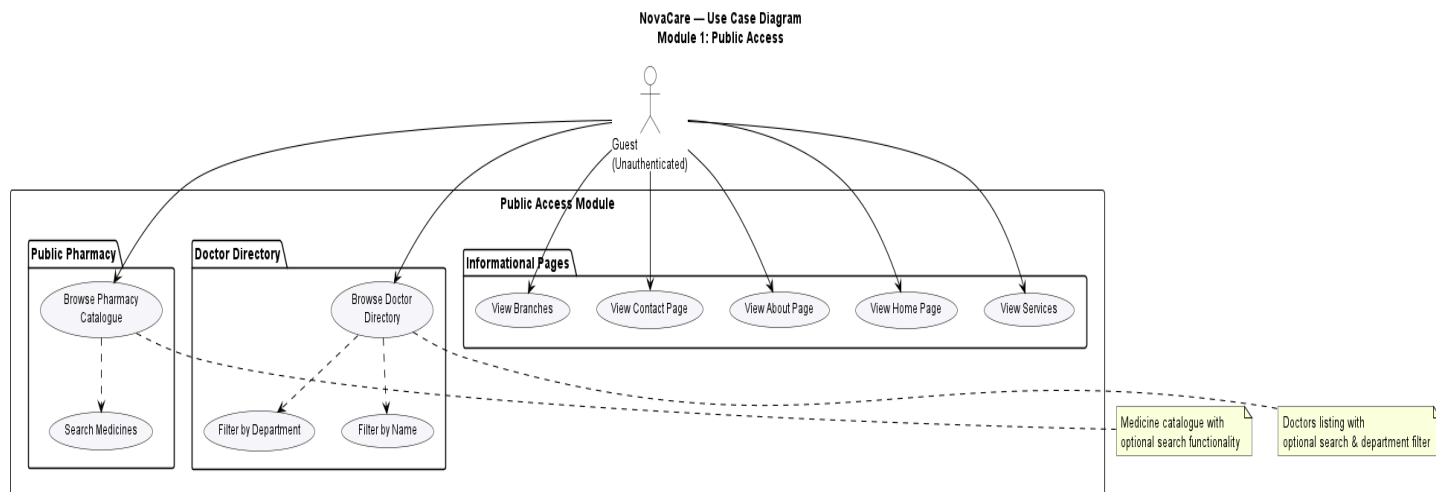
4. SYSTEM DESIGN - UML DIAGRAMS

- Use Case Diagram

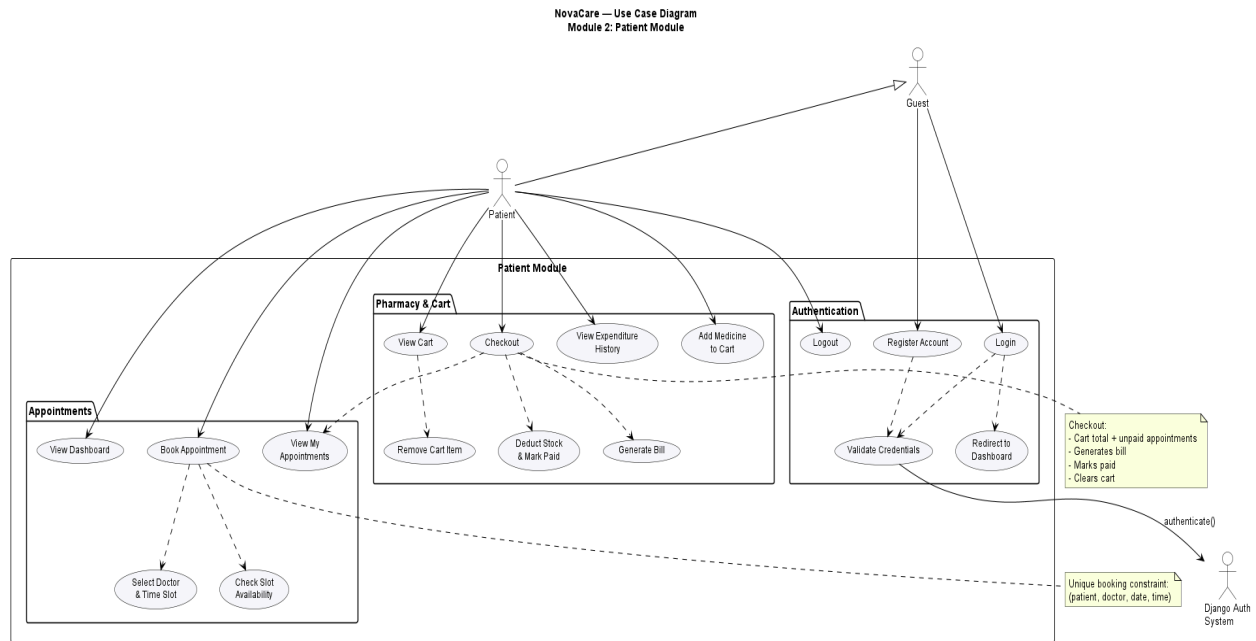
1. Admin Use - Case:



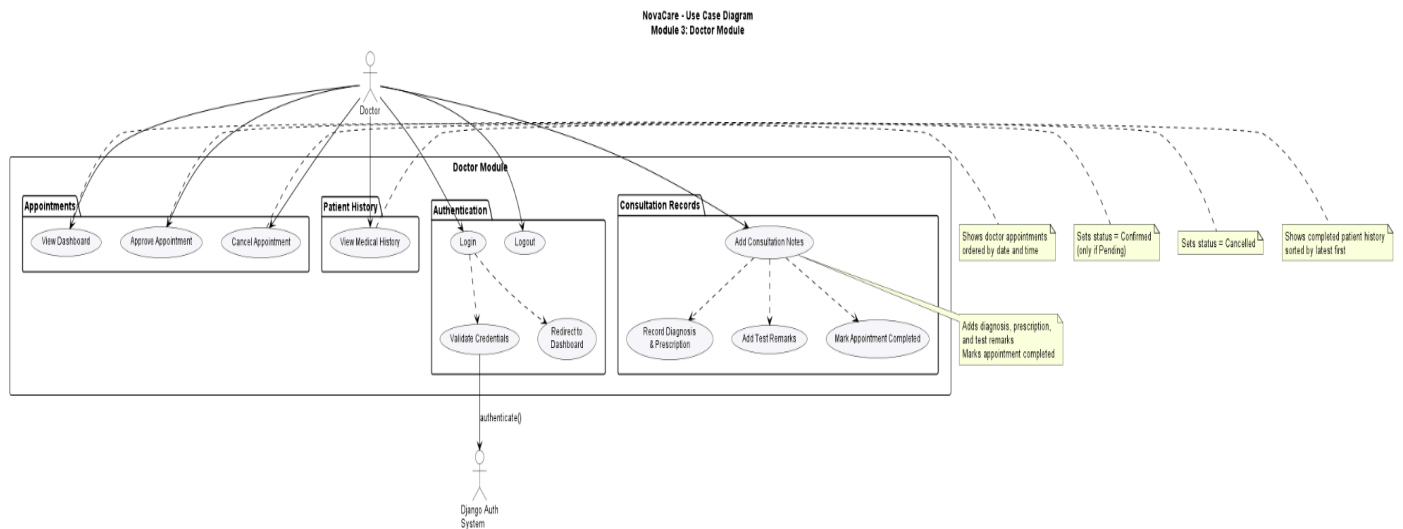
2. Public_Access Use - Case:



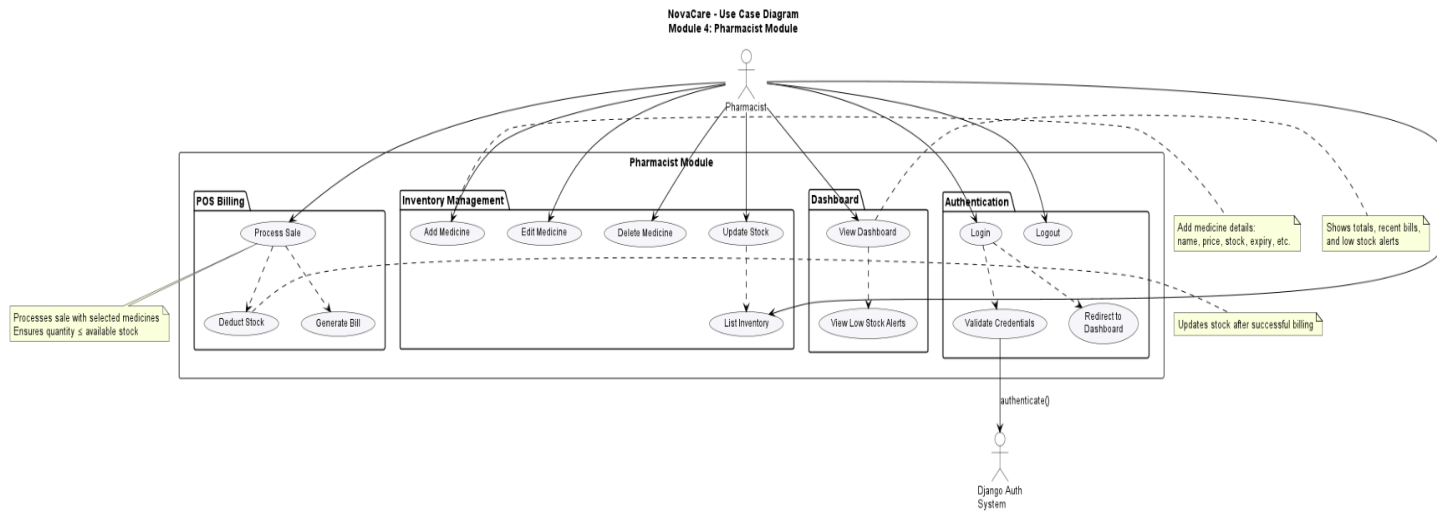
3. Patients Use - Case:



4. Doctor Use - Case:

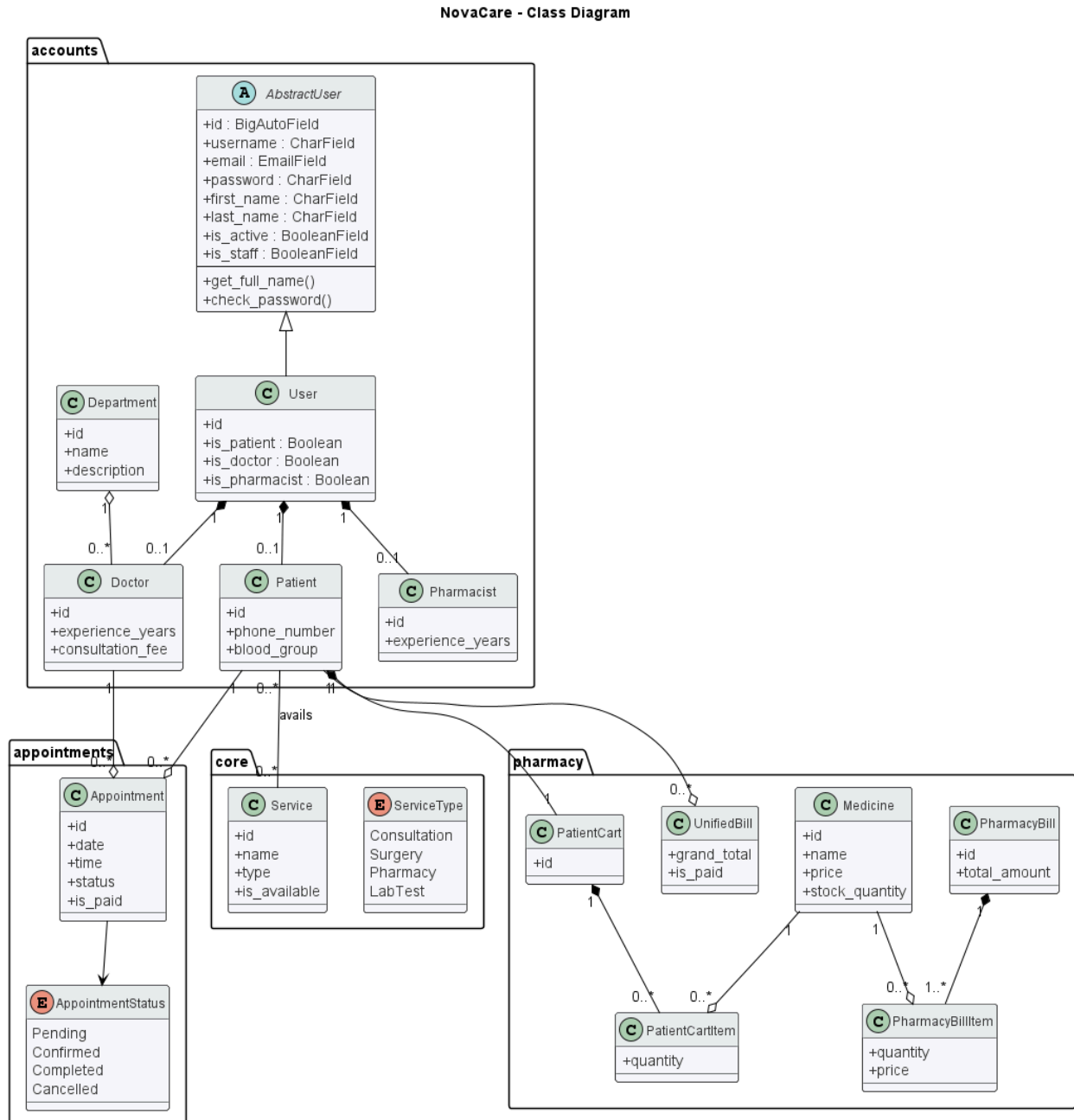


5. Pharmacist Use - Case:



4. SYSTEM DESIGN - UML DIAGRAMS

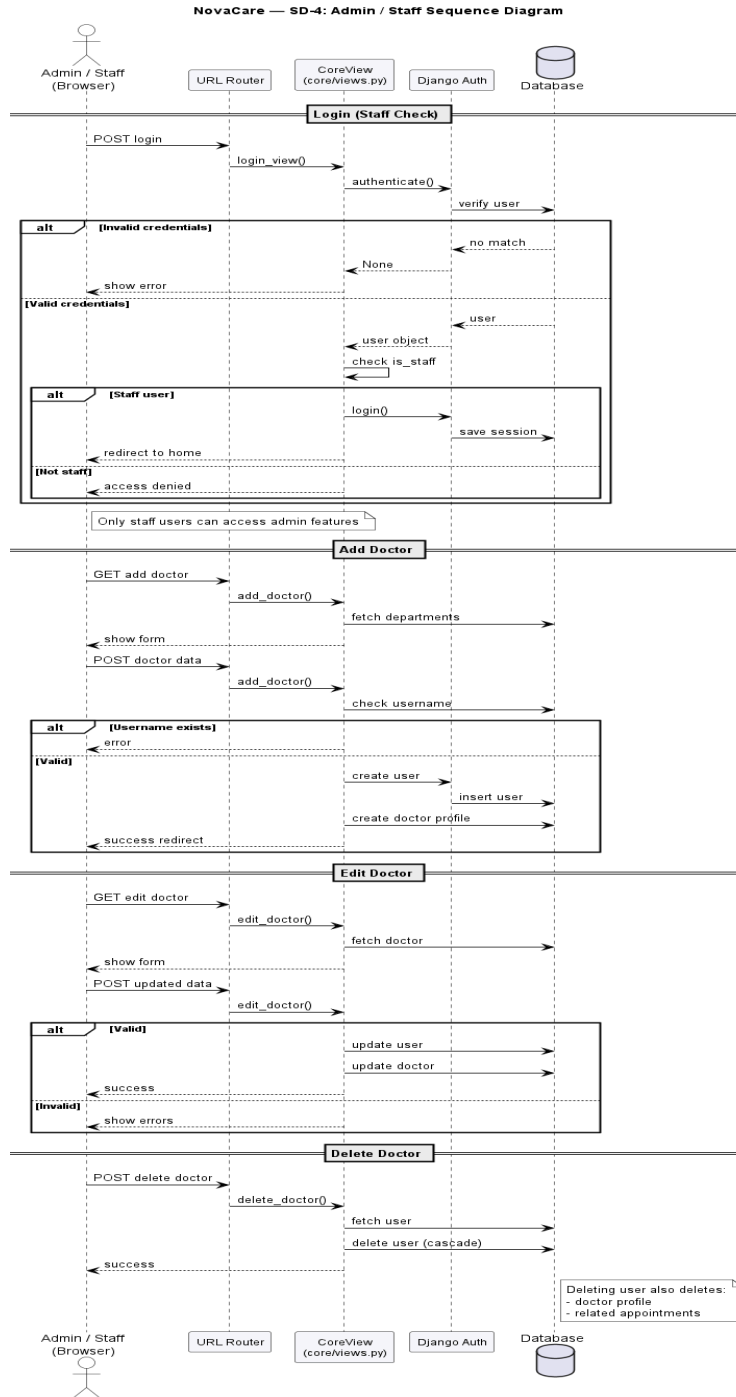
- **Class Diagram:**



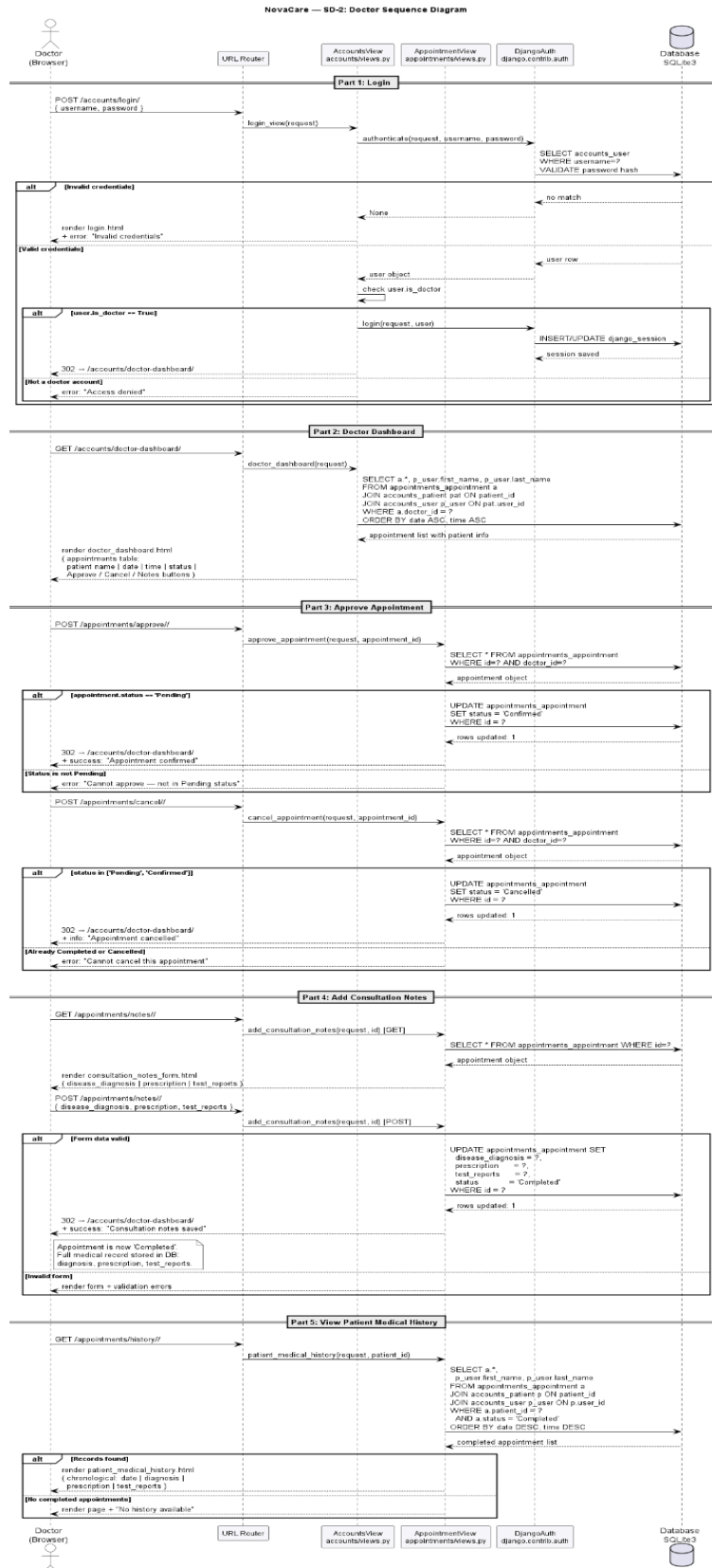
4. SYSTEM DESIGN - UML DIAGRAMS

- **Sequence Diagram:**

1. **Admin:**



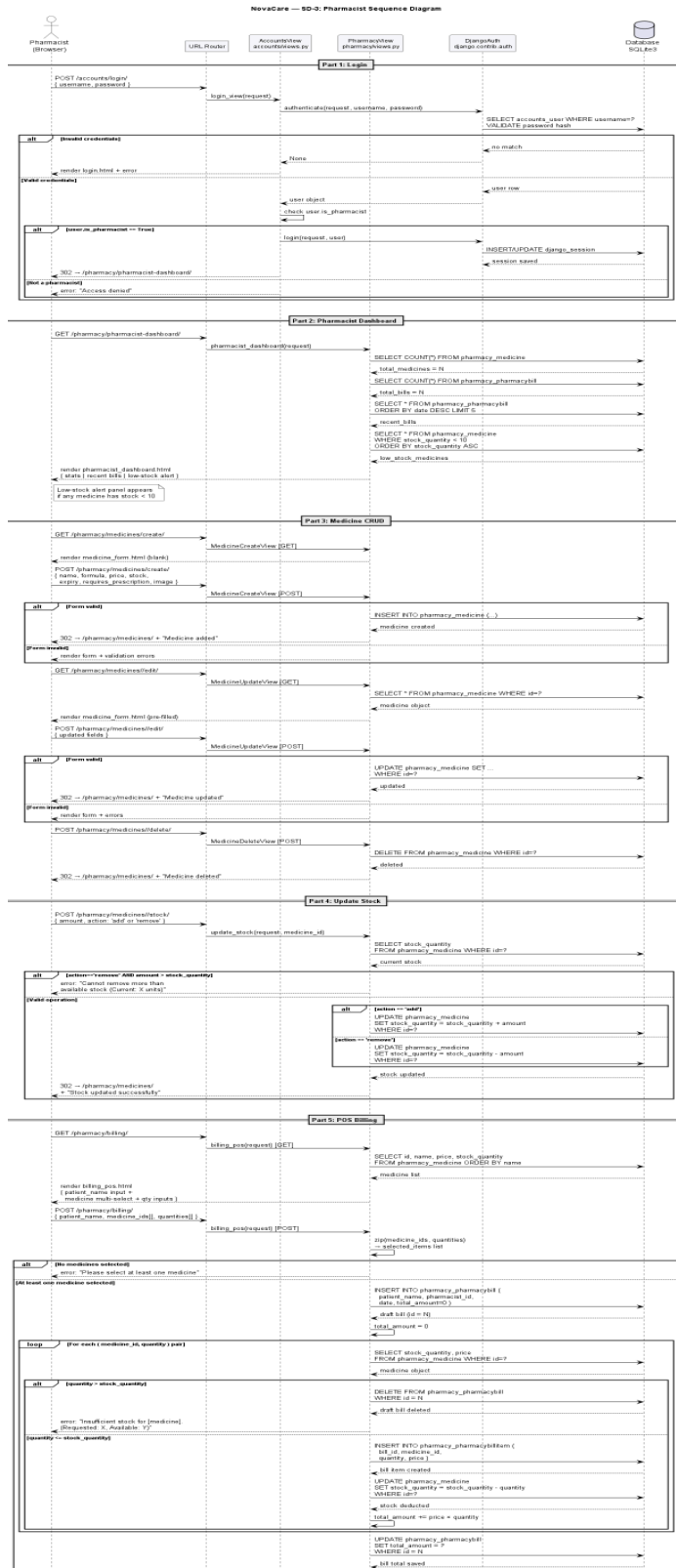
2. Doctor:



3. Patient:

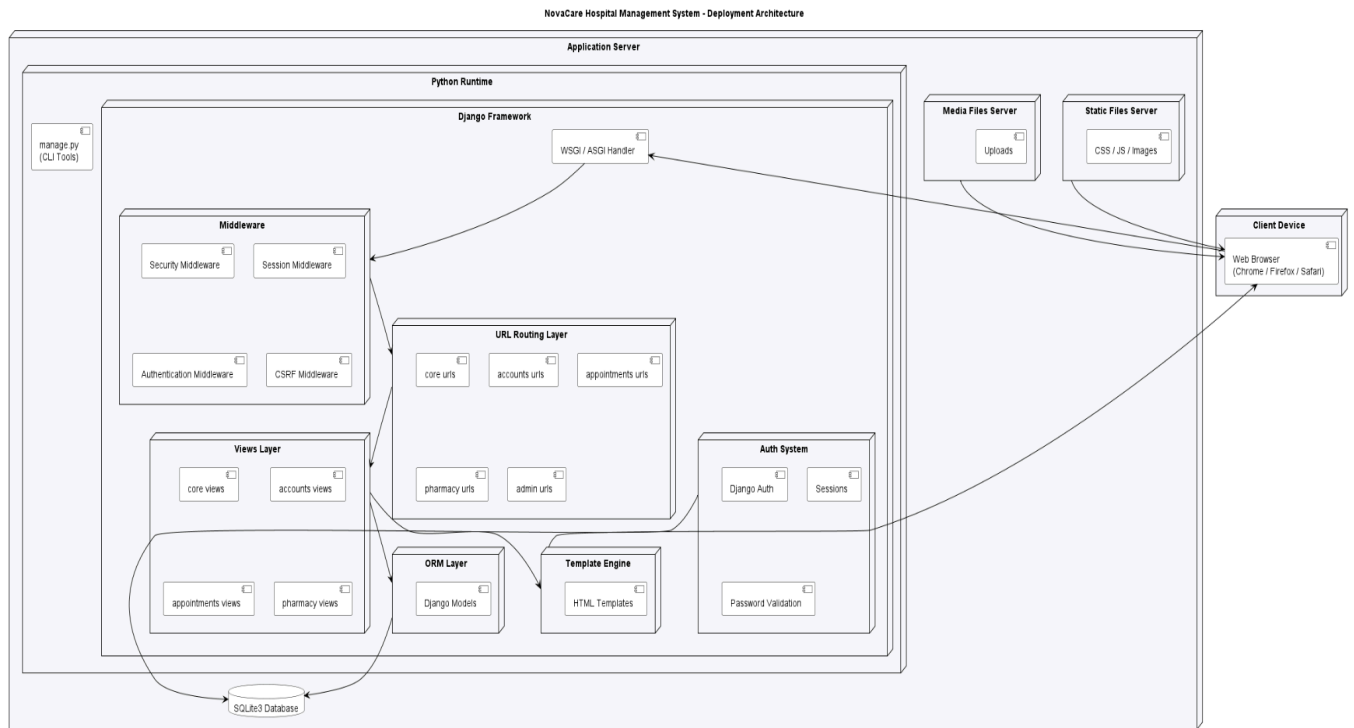


4. Pharmacist:



4. SYSTEM DESIGN - UML DIAGRAMS

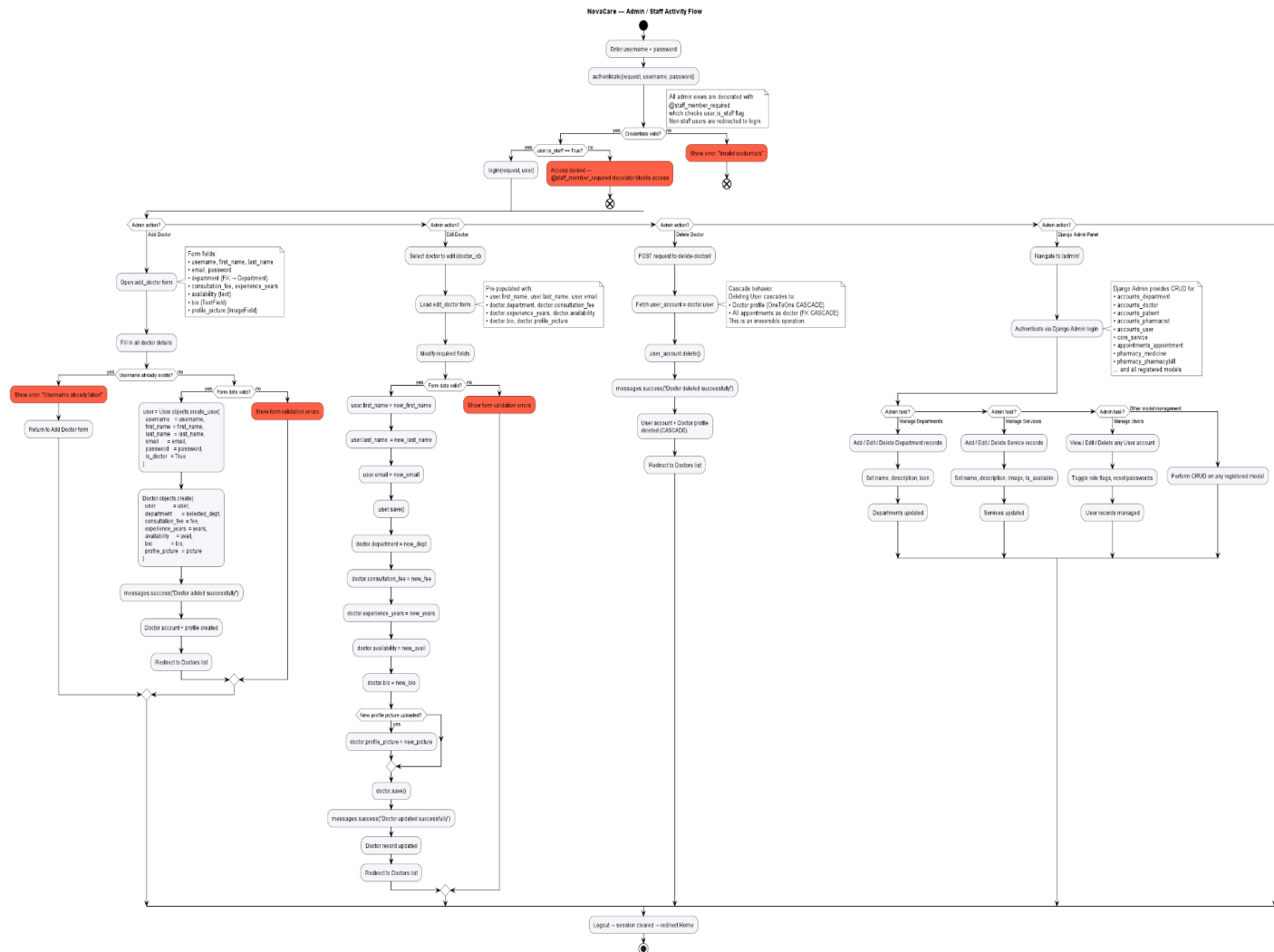
- Deployment Diagram:



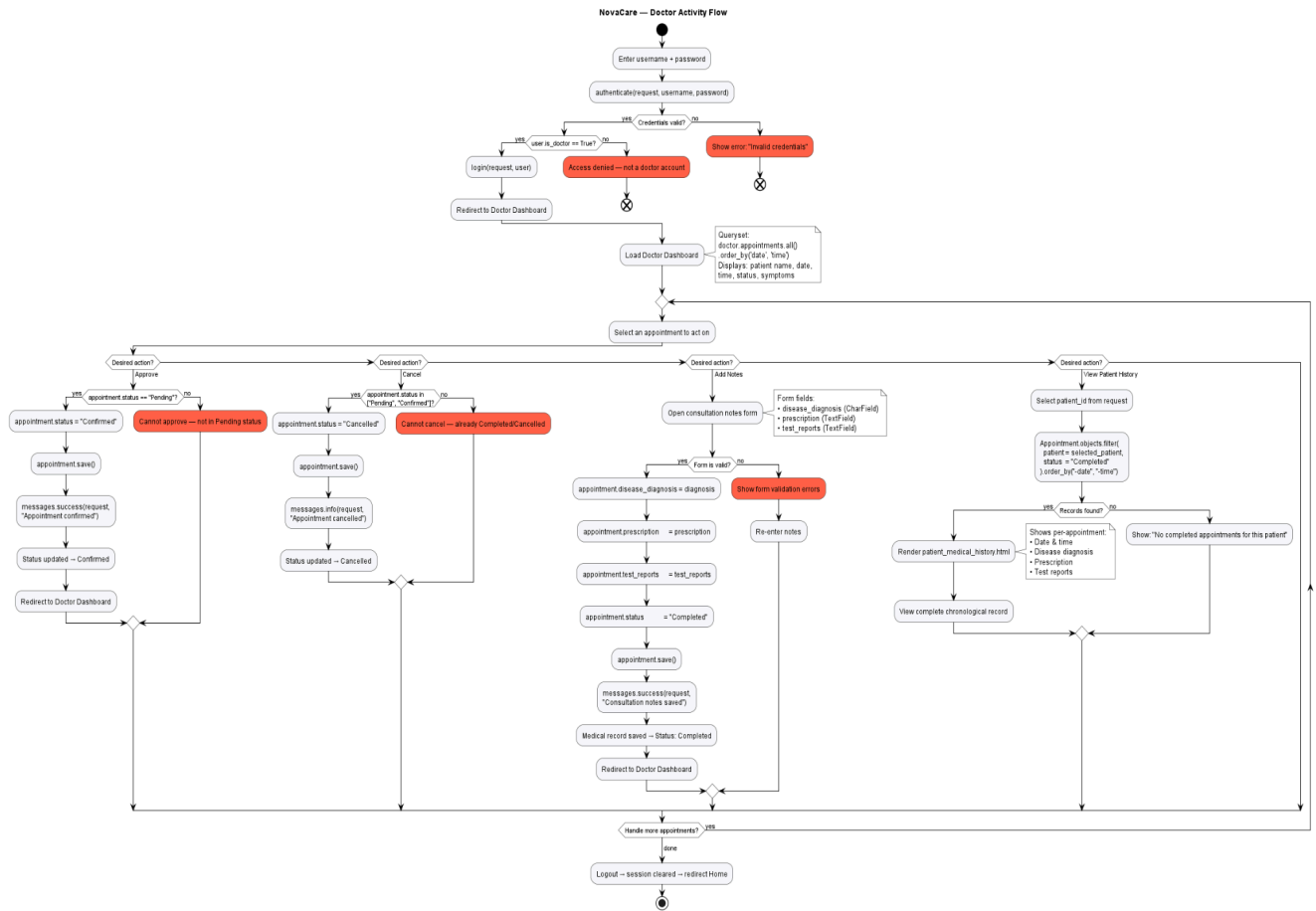
4. SYSTEM DESIGN - UML DIAGRAMS

• Activity Diagram:

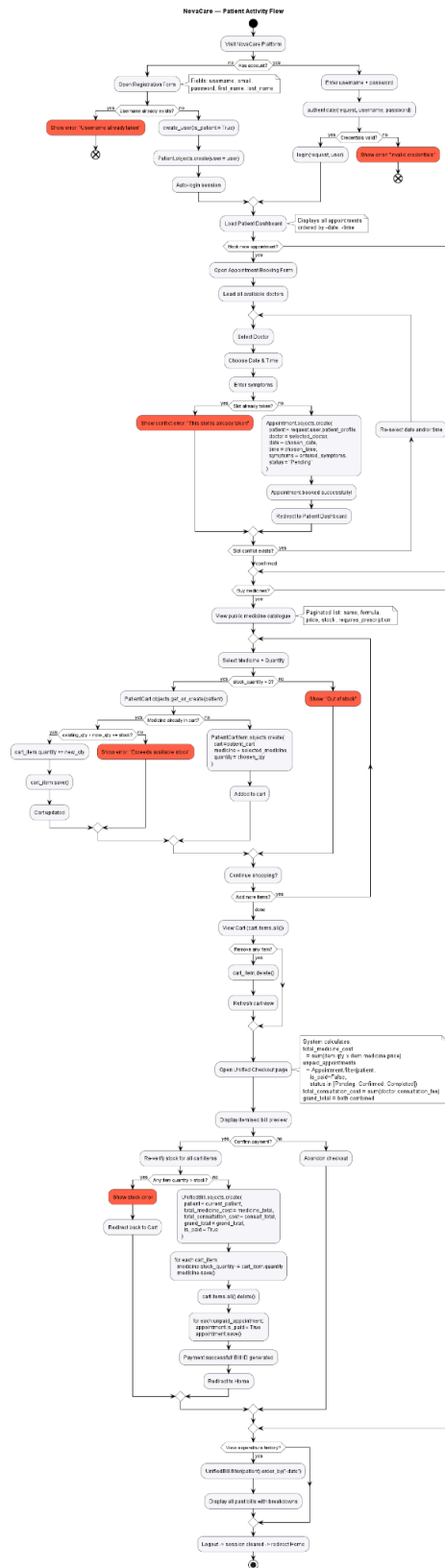
1. Admin Flow:



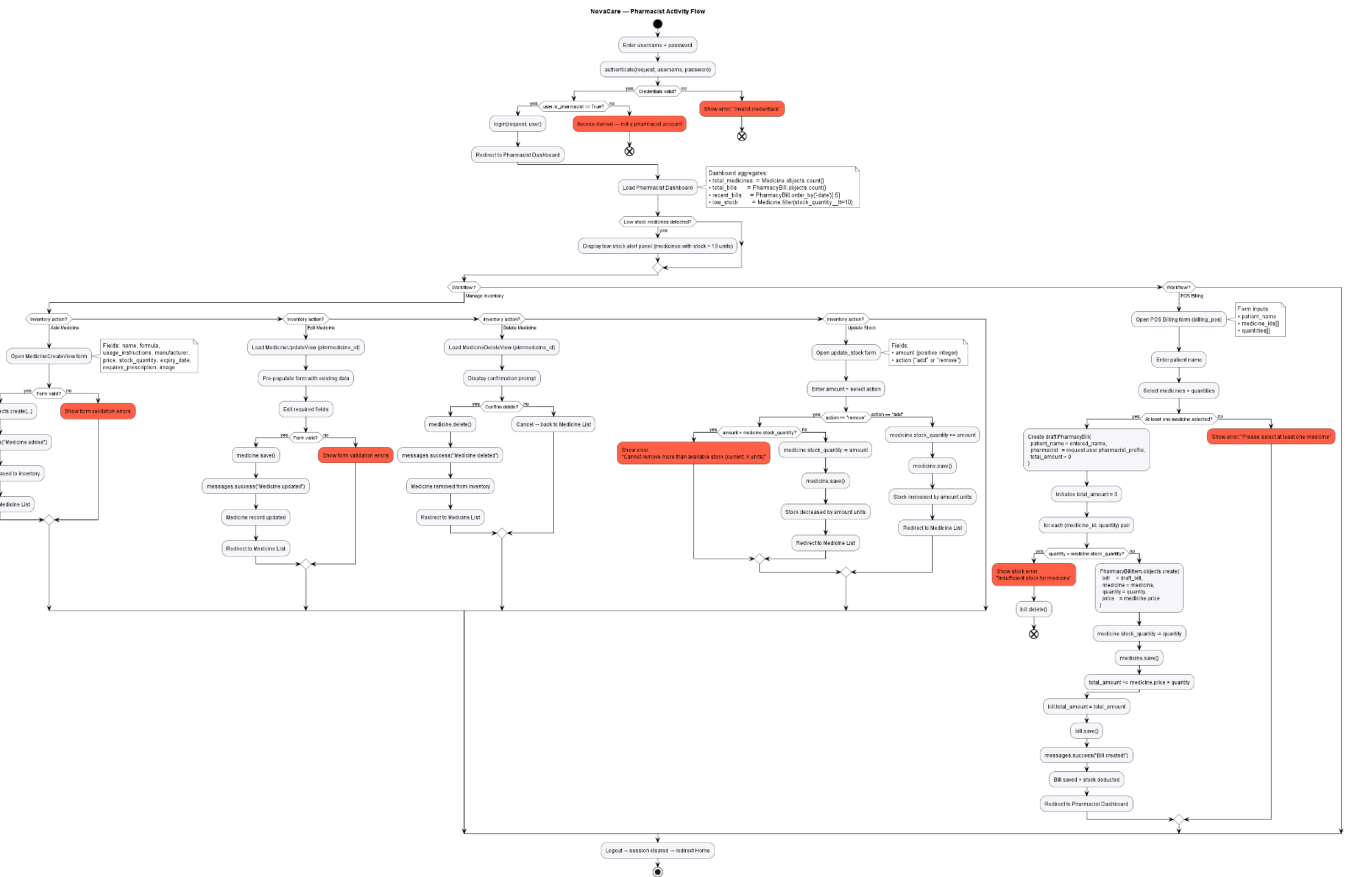
2. Doctor Flow:



3. Patient Flow:

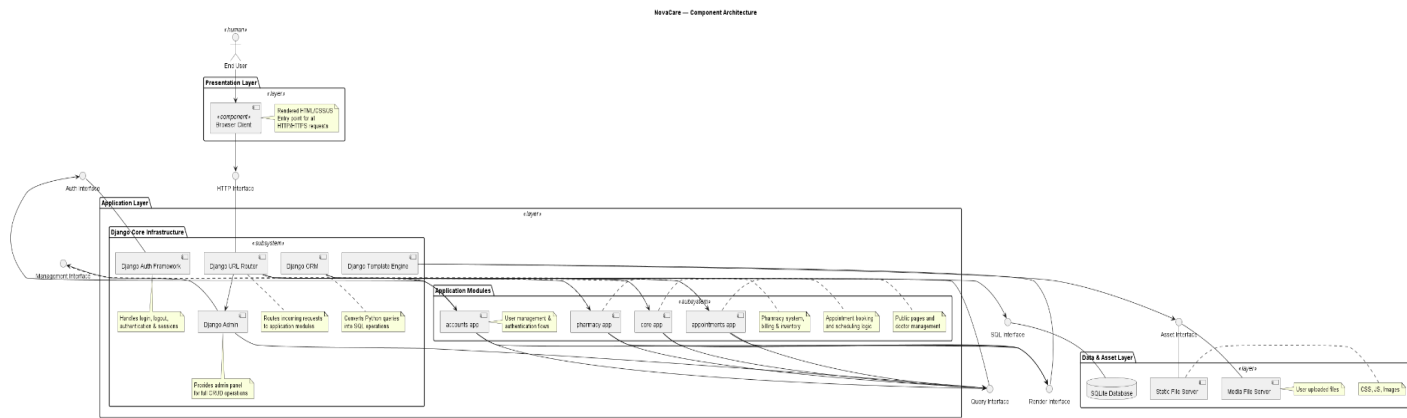


4. Pharmacist Flow:



4. SYSTEM DESIGN - UML DIAGRAMS

- **Component Diagram:**



5. Diagram Used & Their Purpose:

1. Purpose of 4+1 View Model:

- Used to **describe software architecture from multiple perspectives**
- Helps different stakeholders understand the system (users, developers, testers, managers)
- Reduces complexity by splitting system into 5 views:
 - Logical, Development, Process, Physical + Use case view
- Ensures **consistency + completeness of system design**

2. Purpose of Use Case Diagram:

In Unified Modeling Language:

- Shows what the system does from the user's perspective
- Identifies actors (users/systems) and their interactions
- Defines functional requirements of the system
- Helps in understanding system scope and boundaries
- Used for communication between stakeholders and developers

5. Purpose of Diagrams

3. Purpose of Deployment Diagram:

- *Shows physical deployment of software on hardware*
- *Maps software components to nodes (servers, devices)*
- *Helps in understanding system architecture in real environment*
- *Useful for system administrators and DevOps*
- *Used for performance, scalability, and infrastructure planning*

4. Purpose of Component Diagram:

- *Shows organization of system into components/modules*
- *Represents how software is divided and connected*
- *Helps in system modularization and reuse*
- *Shows dependencies between components*
- *Useful for developers to manage large systems*

5. Purpose of Activity Diagram:

- *Represents workflow or business process*
- *Shows step-by-step flow of actions*
- *Includes decision points, loops, and parallel processes*
- *Used to model dynamic behavior of a system*
- *Helpful in understanding logic of a process*

6. Purpose of Sequence Diagram:

- *Shows interaction between objects over time*
- *Displays message flow in chronological order*
- *Used for modeling specific scenarios/use cases*
- *Helps understand how system behaves dynamically*
- *Clarifies object responsibilities*

7. Purpose of Class Diagram:

A class diagram is used to:

- *Show the structure of a system*
- *Represent classes, attributes, and methods*
- *Define relationships between classes*

5. Summary

Diagrams:

1. *4+1 View Model*
2. *5 UseCase Diagrams*
3. *1 Deployment*
4. *4 Sequence Diagrams*
5. *1 Class Diagram*
6. *1 Component Diagram*
7. *4 Activity Diagrams*